

Exploit-Oriented Pre-Deployment Scanner

Comparative technical evaluation against Slither

Author: Niko — Date: 20.06.2026 — Version: v0.1



Executive summary

This report compares a lightweight, exploit-oriented pre-deployment scanner with Slither. Objective: assess noise reduction, exploit relevance prioritisation, developer accessibility and pre-deployment usability. The custom scanner aims to surface actionable exploit signals for developers without requiring deep familiarity with static analysis tooling.

📄 All screenshots and raw proofs: **Proofs.pdf**

Test methodology

01

Environment

Solidity contract: VulnerableVault (hh.sol) — Solidity ^0.8.0.
Slither run via CLI; custom scanner executed in browser.
Both run against identical source.

02

Evaluation criteria

1) Detection of exploitable vulnerabilities 2) Signal-to-noise ratio 3) Structural vs exploit-oriented reporting 4) Clarity and developer relevance of output



Slither — raw findings (selected)

Command: `slither hh.sol`

reentrancy-eth

Reentrancy in
VulnerableVault.withdra
w(uint256). External
low-level call with value
then state write:
balances[msg.sender] -=
amount.

missing-zero-check

changeOwner(address).n
ewOwner lacks
zero-address validation
— best-practice warning.

solc-version

Version constraint ^0.8.0
flagged for historical
compiler issues
(informational).

low-level-calls

Low-level call detected
in withdraw(uint256):
msg.sender.call{value:
amount}()

Custom scanner — output summary

2205ms

Total scan time

scan_1771362017003 —
2205.25ms

4

Functions analysed

Four functions processed
with lightweight heuristics.

1

Vulnerabilities

One critical reentrancy
signal (external call with
value).

1

Warnings

One unchecked call (transfer
without success check).



Custom scanner — detailed signals

- **CRITICAL — Reentrancy**

External call with value transfer at line 20. flagged as exploit-relevant due to state write following call. Severity: critical; action: review for mutex or pull-payments.

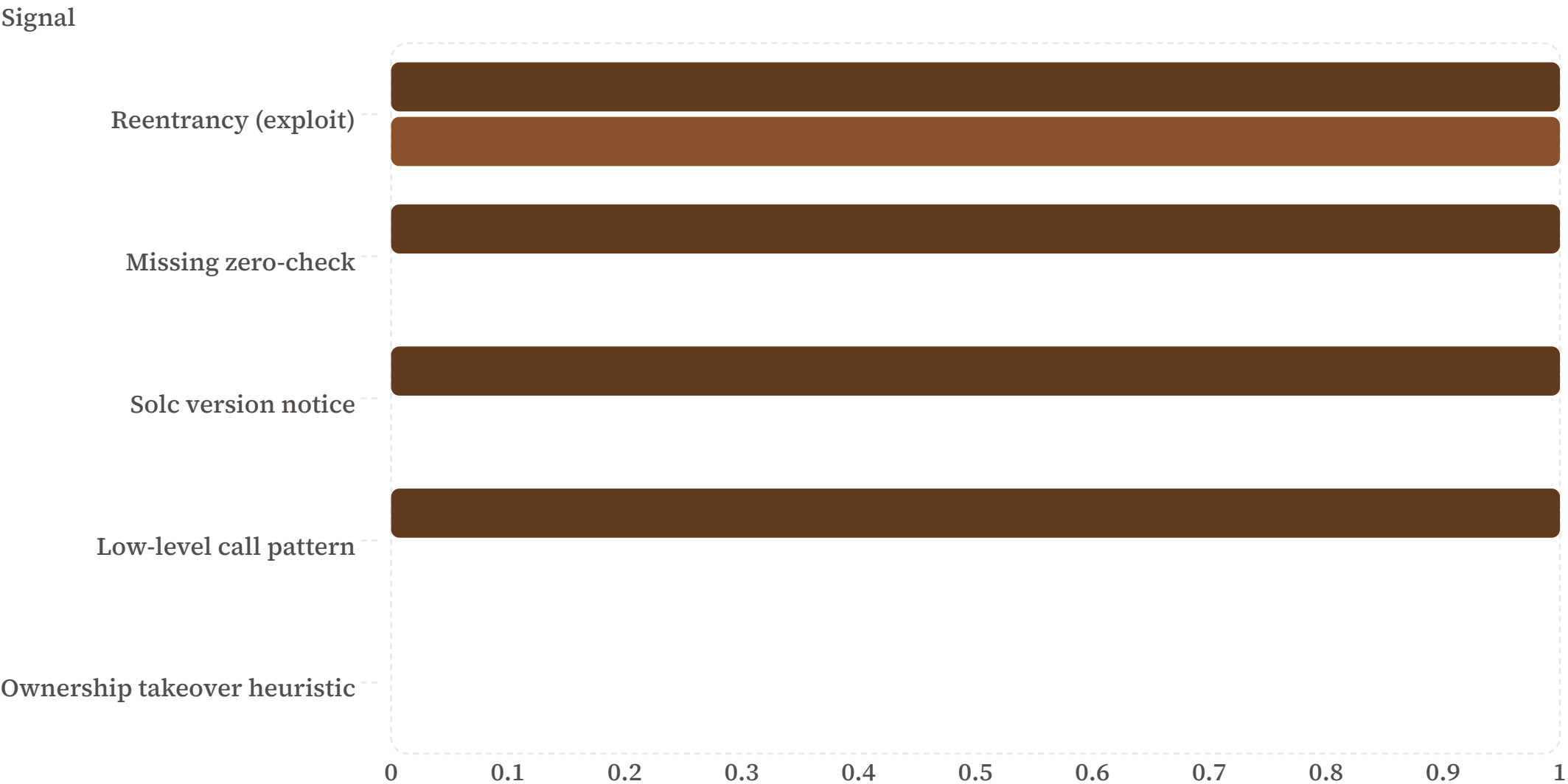
- **WARNING — UNCHECKED_CALL**

Transfer without a success check at line 32. Risk: silent failures may lead to inconsistent state; recommend explicit require on call return value.



Comparative analysis — concise matrix

Slither Custom



Interpretation: Slither reports structural patterns broadly; the custom scanner concentrates on exploit-relevant signals and reduces informational noise, but omits several structural warnings.

Observations & development gaps

1 Different focuses

Slither emphasises structural and best-practice detections. The custom scanner prioritises value-transfer exploit patterns for developer actionability.

2 False negatives / gaps

Custom scanner missed zero-address and low-level call pattern flags; access control and ownership heuristics are immature.

3 Usability

Custom scanner output is concise and developer-friendly for pre-deployment checks; useful for quick triage before deeper audits.

Positioning, limitations & roadmap

🛡️ POSITIONING

Pre-audit filter — lightweight, fast, developer oriented.

Current limitations

- No advanced access-control inference
- No cross-function exploit path chaining
- Limited function name resolution and mapping

Planned improvements (v0.2)

1. Access control heuristics and intent inference
2. Ownership takeover detection
3. Exploit-path chaining and contextual ranking

Conclusion & recommended actions

Conclusion

The custom scanner successfully detected the critical reentrancy exploit and reduced reporting noise; it complements rather than replaces Slither for pre-deployment triage.

Immediate recommendations

- Address reentrancy: adopt pull-payments or re-order state writes before external calls.
- Add explicit success checks for low-level transfers.
- Run Slither (or a full audit) after quick triage with the custom scanner.

Next steps for scanner

Prioritise access-control heuristics, ownership detection and exploit-path chaining to reduce false negatives and raise precision for critical exploit classes.

